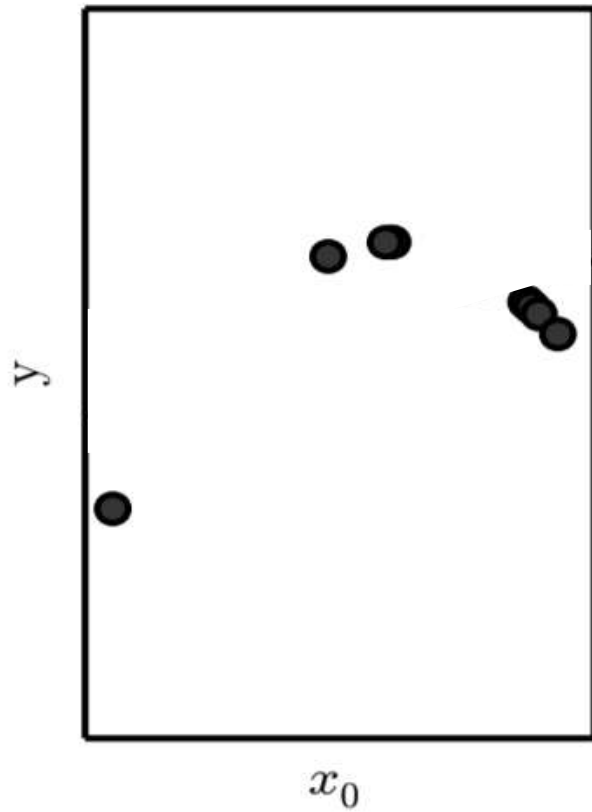
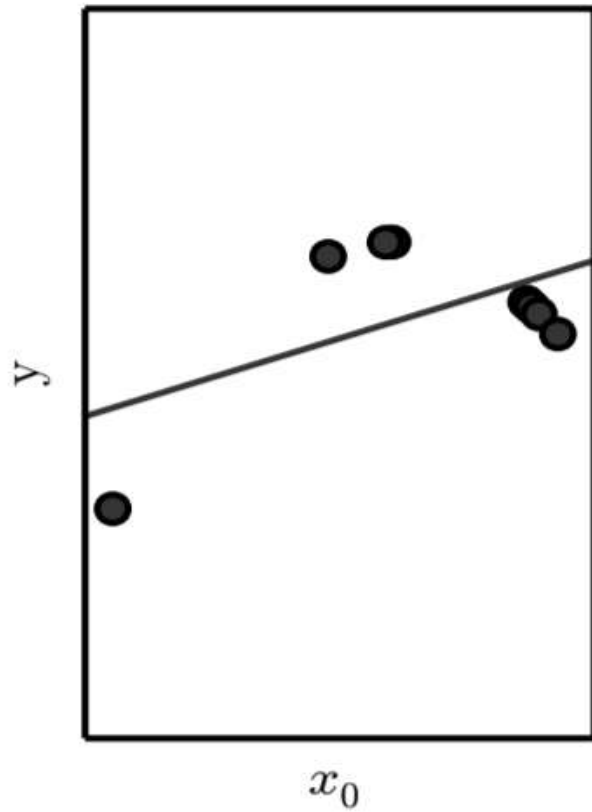


Regularization

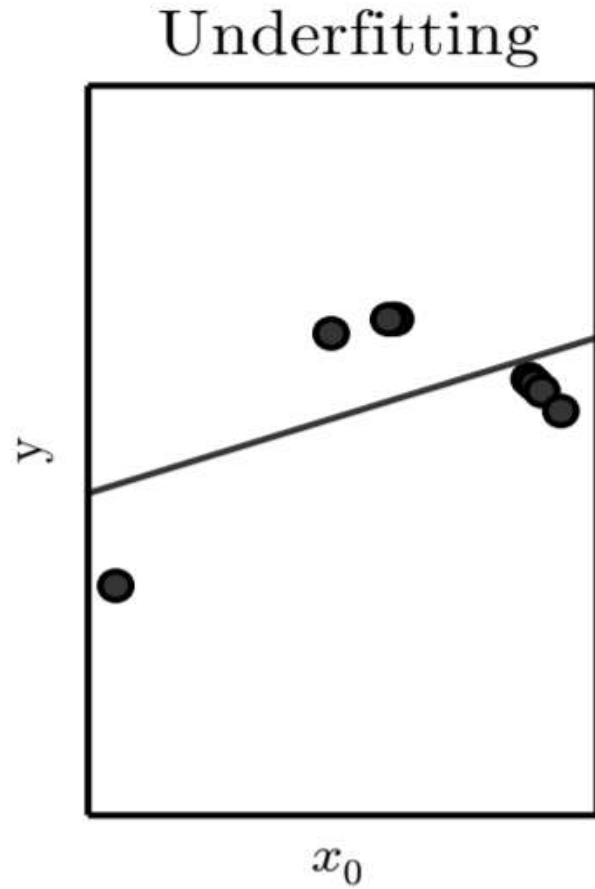
Underfitting and Overfitting in Polynomial Estimation



Underfitting and Overfitting in Polynomial Estimation

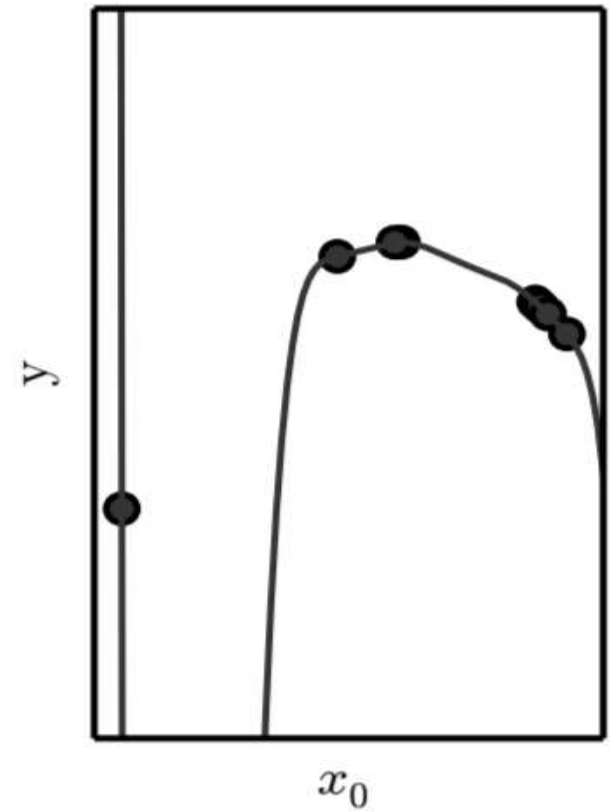
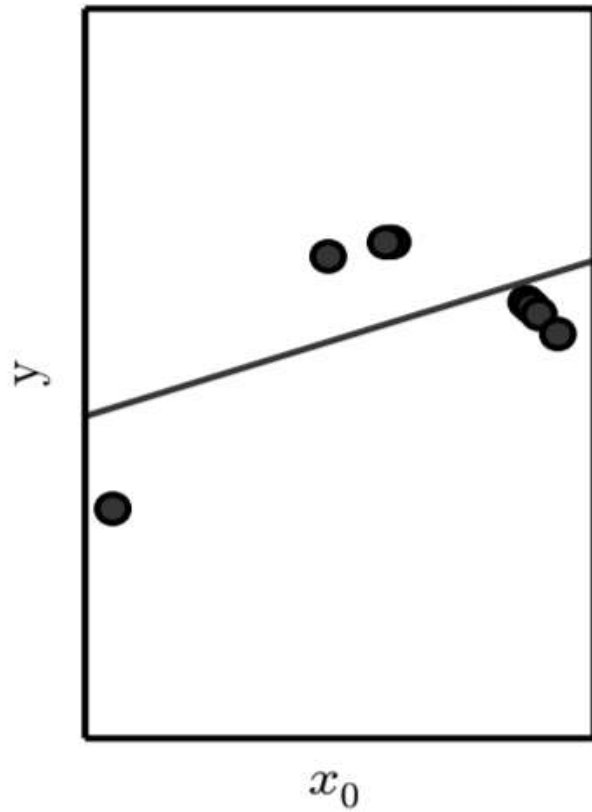


Underfitting and Overfitting in Polynomial Estimation



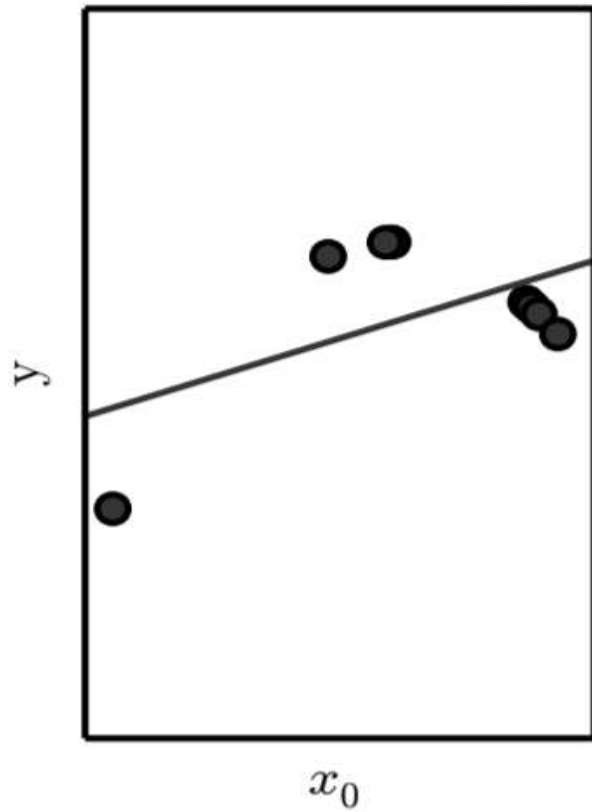
Underfitting and Overfitting in Polynomial Estimation

Underfitting

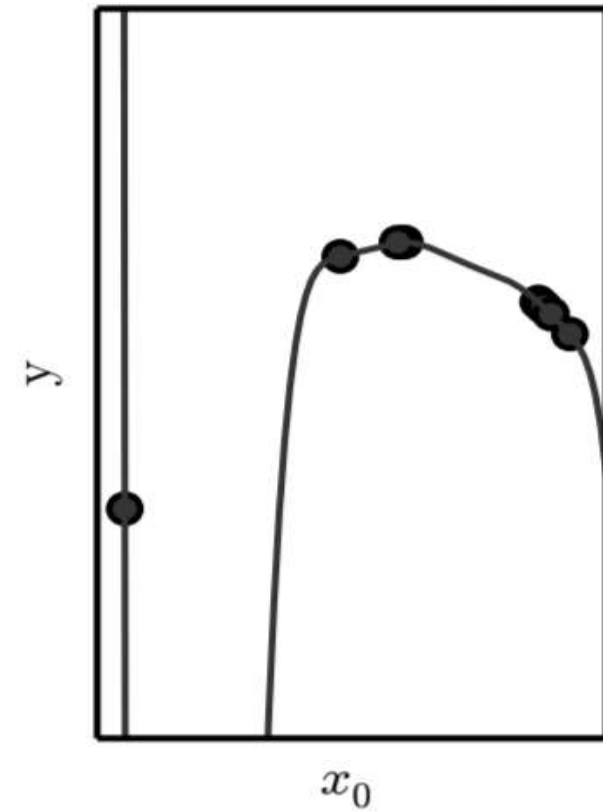


Underfitting and Overfitting in Polynomial Estimation

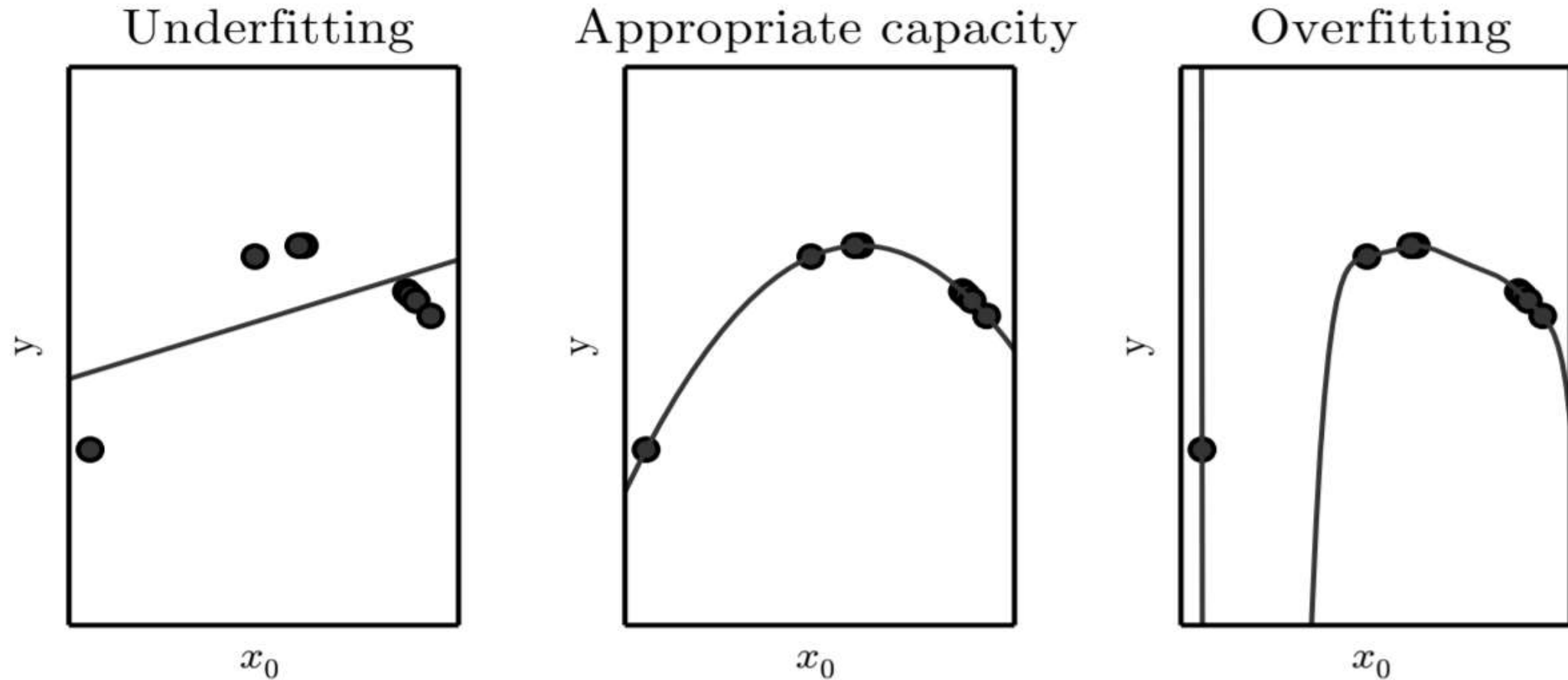
Underfitting



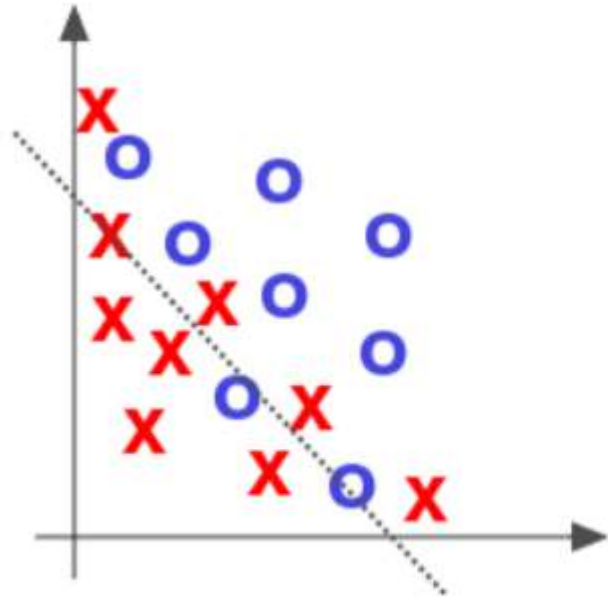
Overfitting



Underfitting and Overfitting in Polynomial Estimation



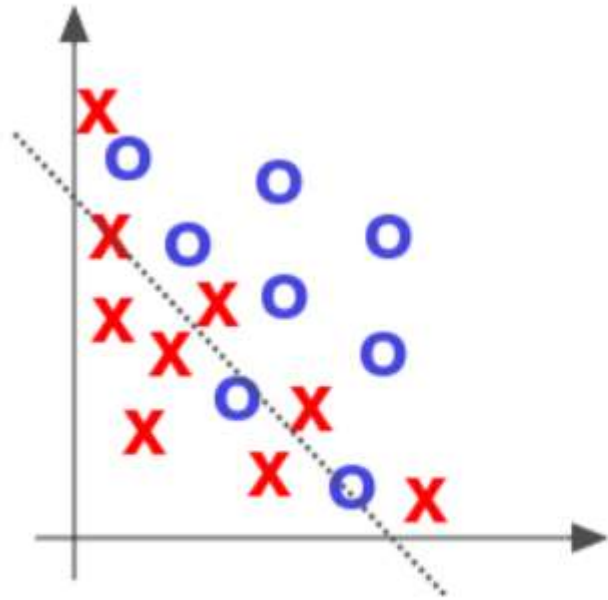
Over- and Underfitting



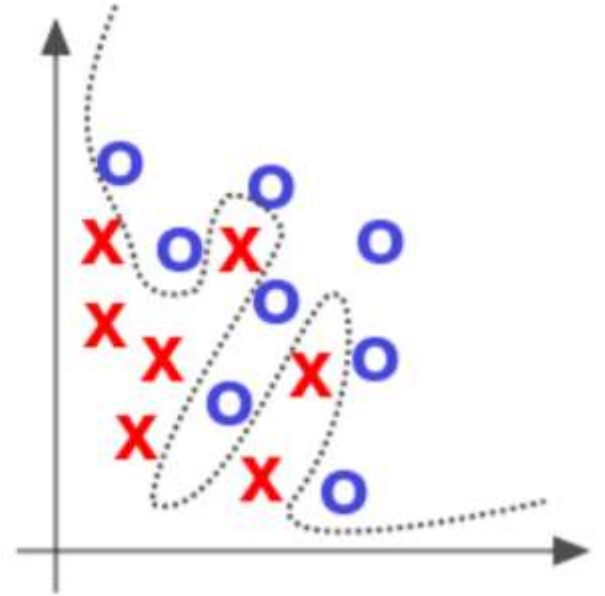
Underfitted

Source: Deep Learning by Adam Gibson, Josh Patterson, O'Reilly Media Inc., 2017

Over- and Underfitting



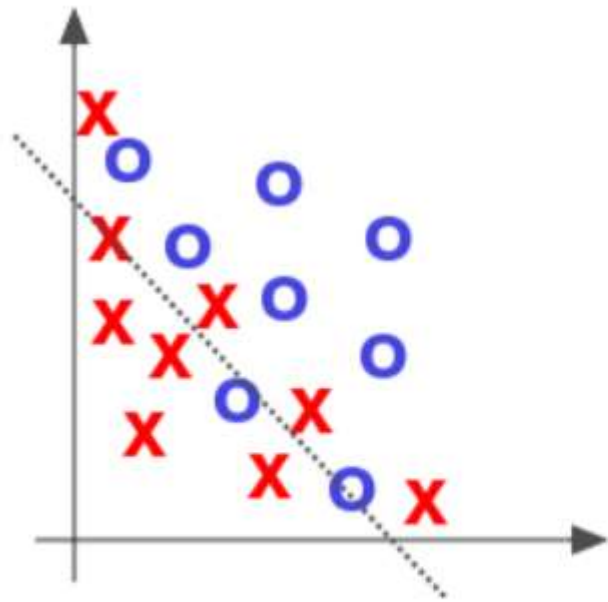
Underfitted



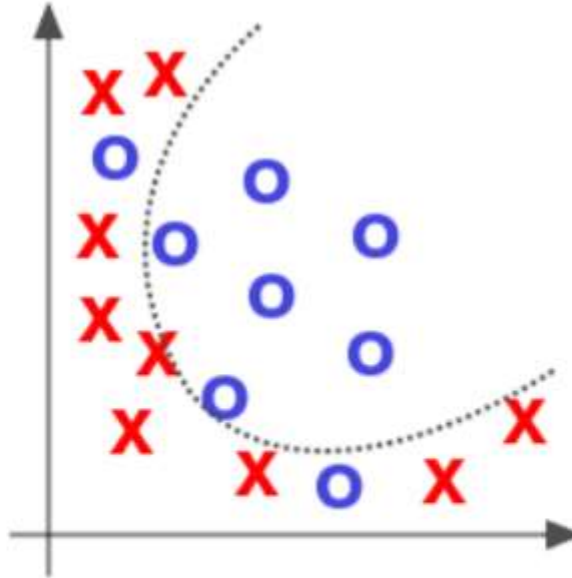
Overfitted

Source: Deep Learning by Adam Gibson, Josh Patterson, O'Reilly Media Inc., 2017

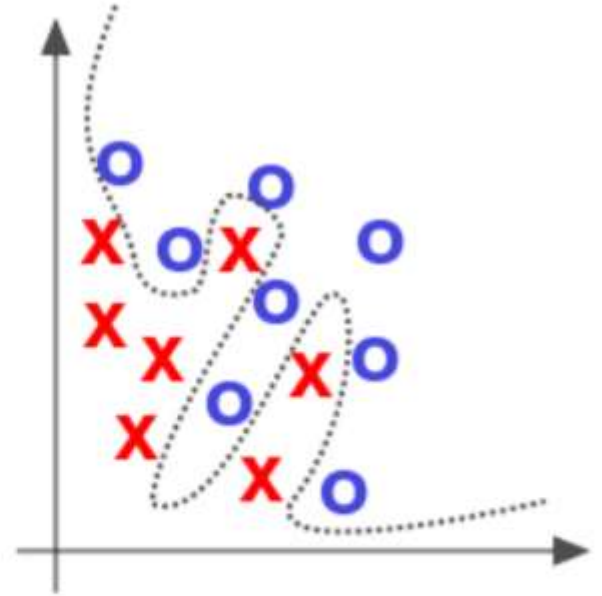
Over- and Underfitting



Underfitted



Appropriate

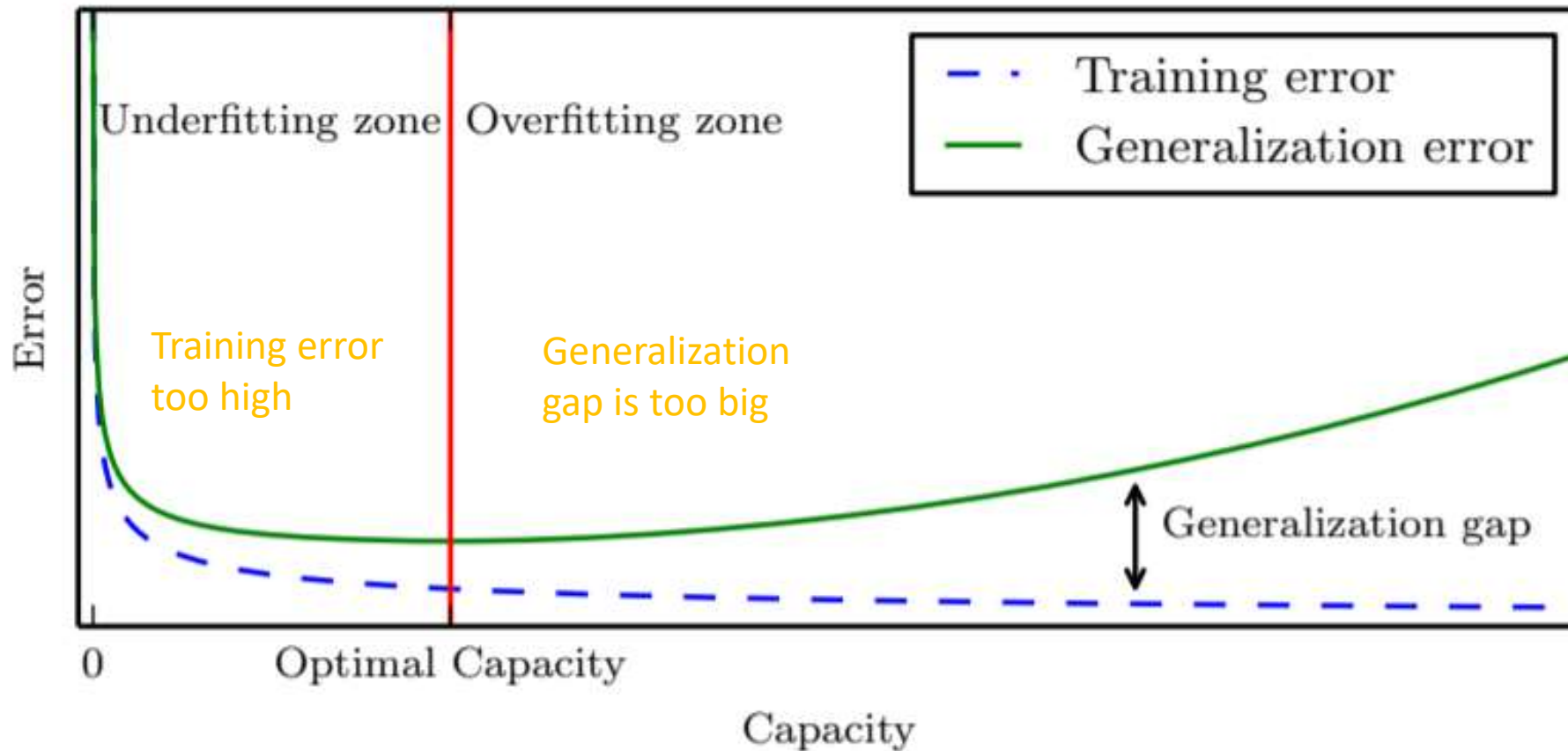


Overfitted

Source: Deep Learning by Adam Gibson, Josh Patterson, O'Reilly Media Inc., 2017

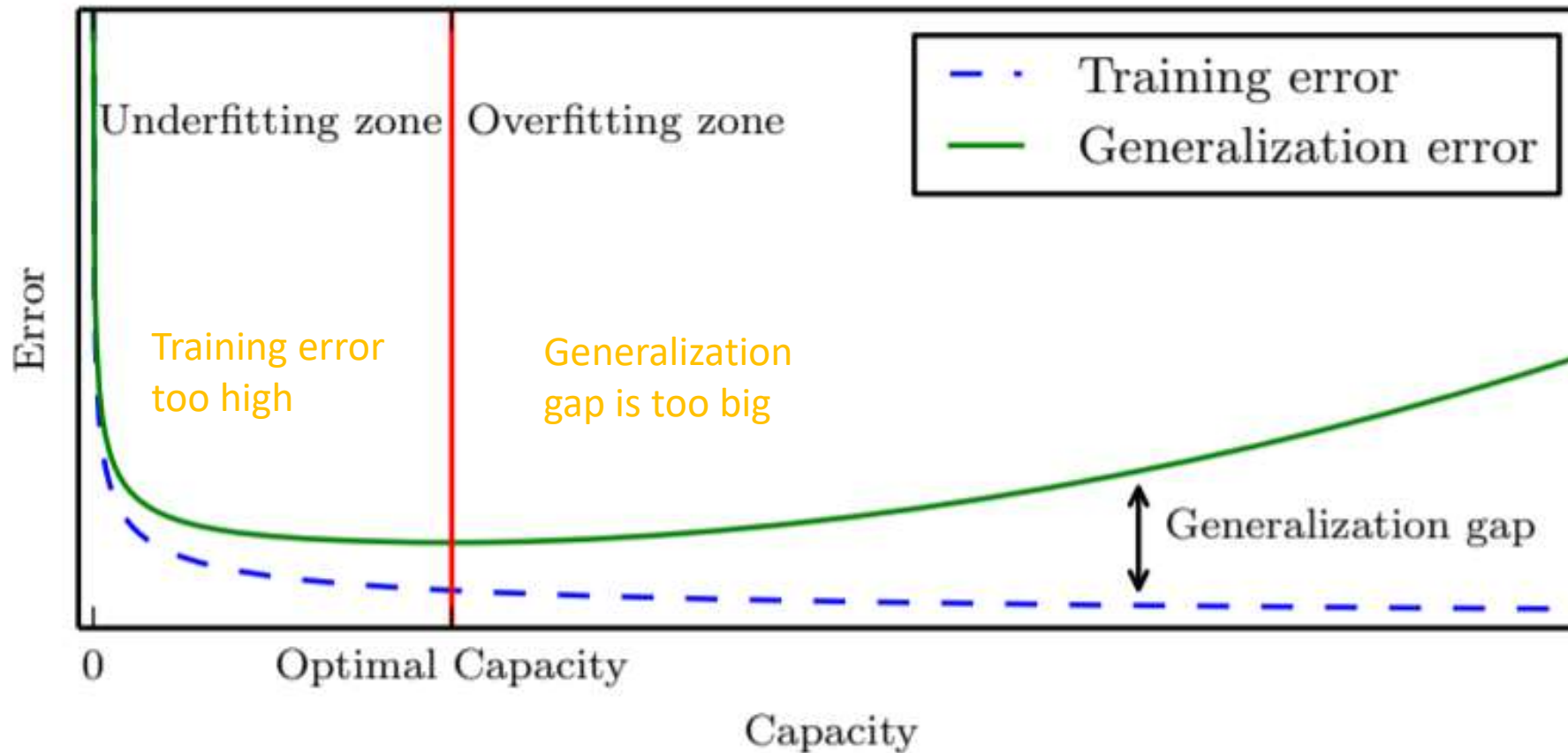
Training a Neural Network

Training/ Validation curve



Training a Neural Network

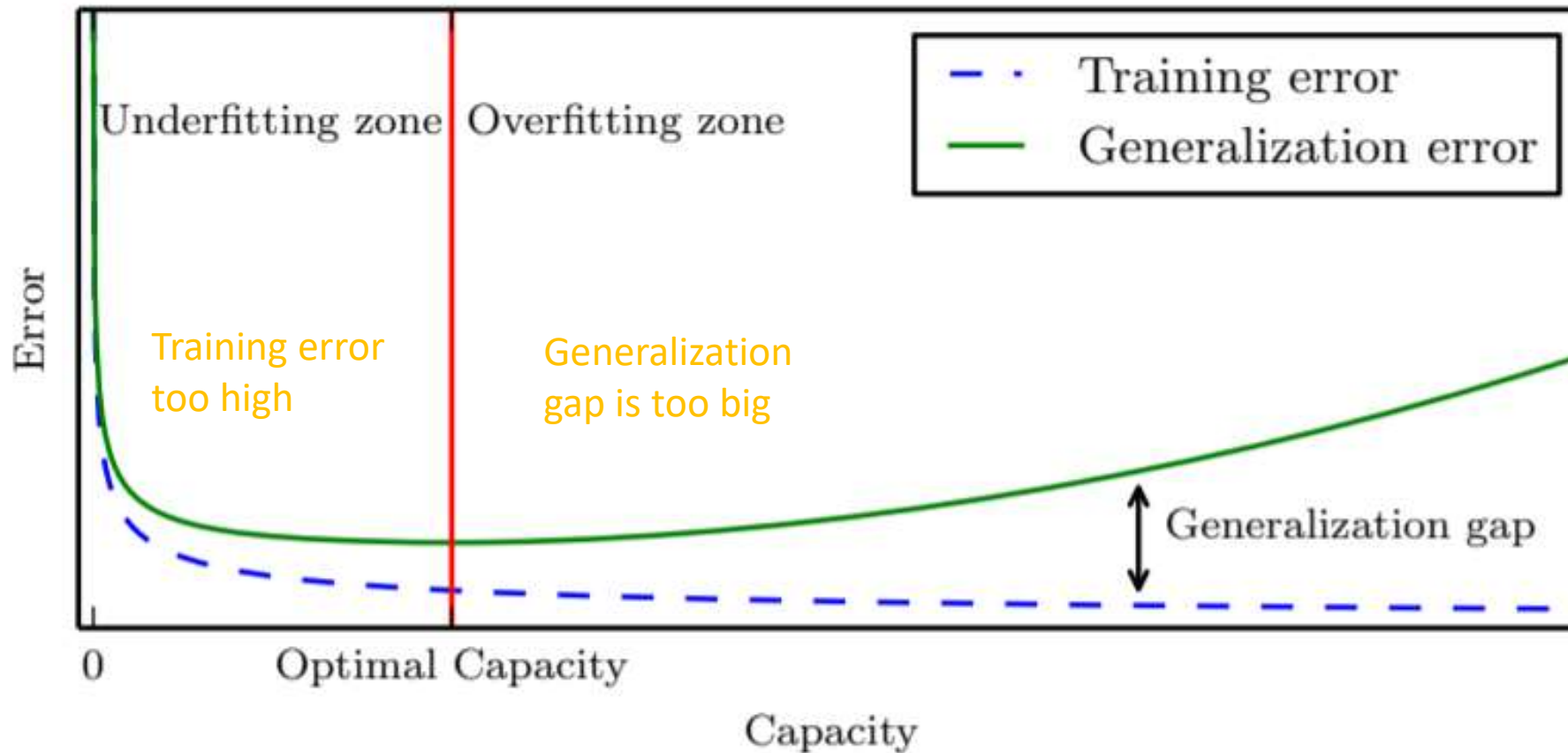
Training/ Validation curve



How can we prevent our model from overfitting?

Training a Neural Network

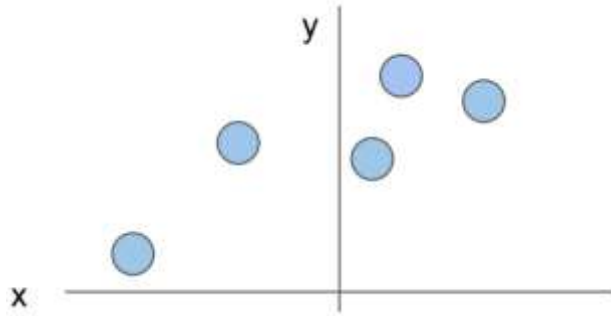
Training/ Validation curve



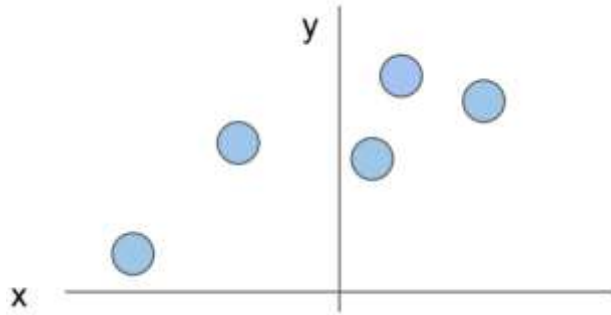
How can we prevent our model from overfitting?

Regularization

Regularization: Prefer Simpler Models



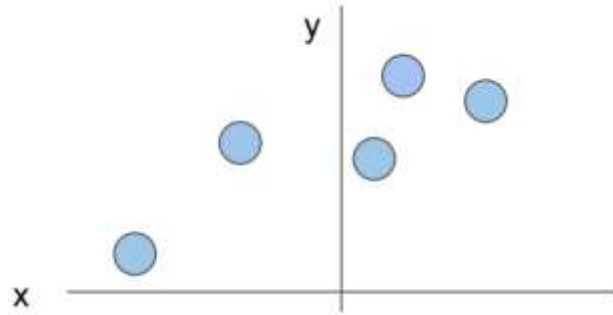
Regularization: Prefer Simpler Models



$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

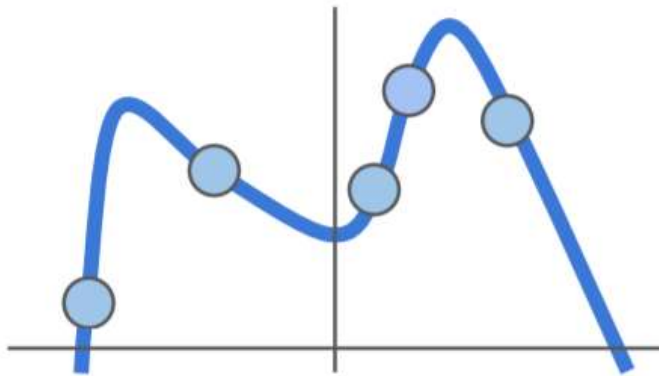
Data loss: Model predictions should match training data

Regularization: Prefer Simpler Models

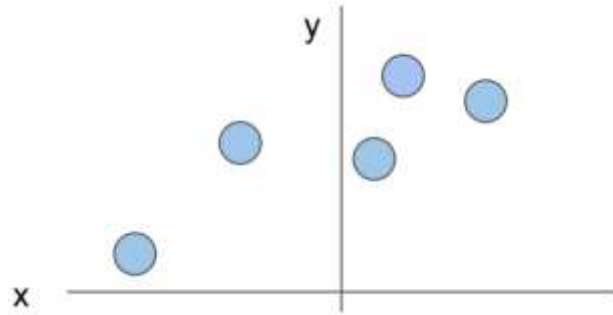


$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data

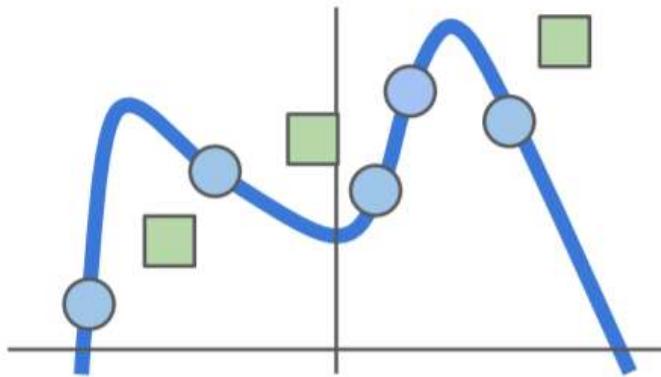


Regularization: Prefer Simpler Models

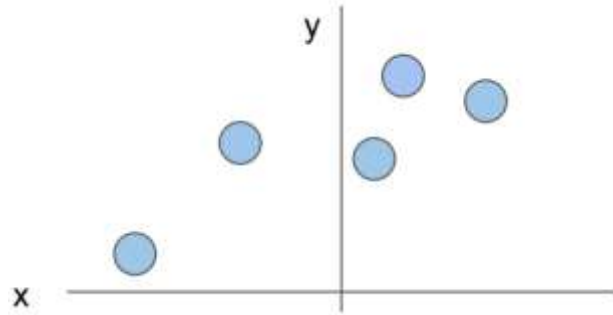


$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data

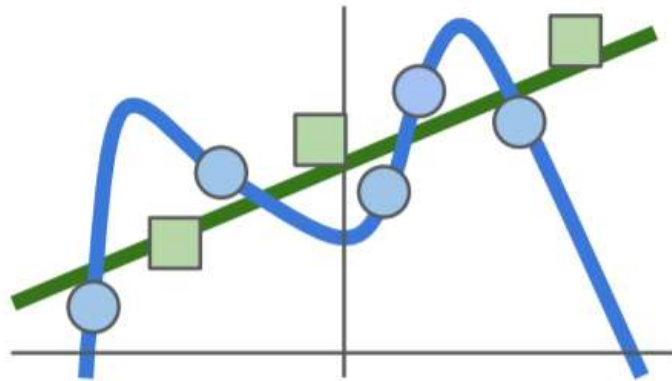


Regularization: Prefer Simpler Models

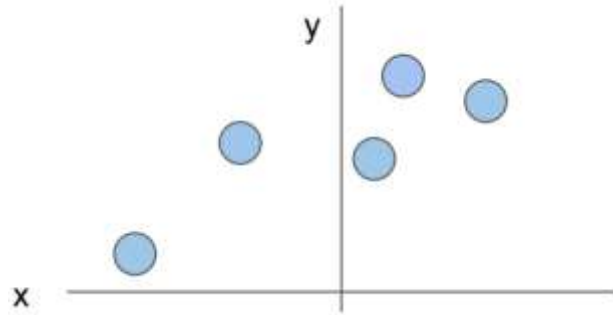


$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Data loss: Model predictions should match training data



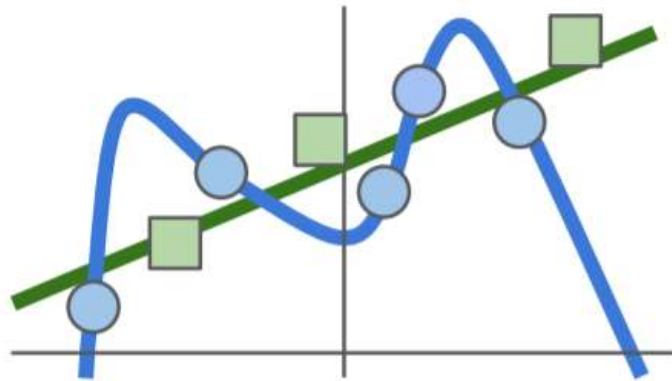
Regularization: Prefer Simpler Models



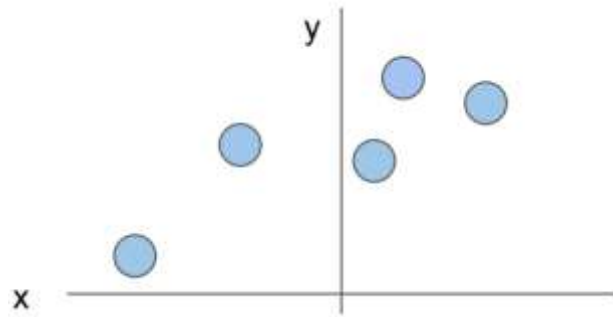
$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data



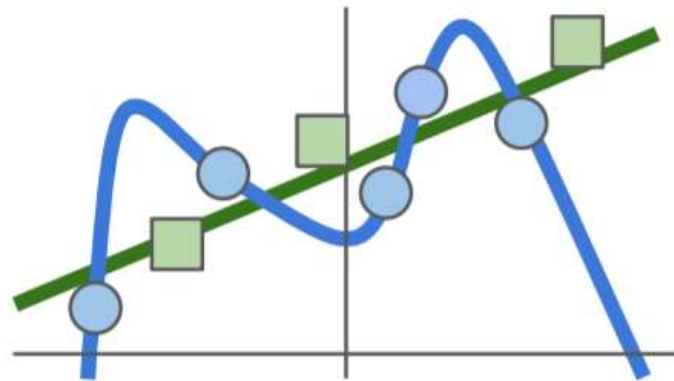
Regularization: Prefer Simpler Models



$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data



Occam's Razor:

"Among competing hypotheses, the simplest is the best"

William of Ockham, 1285 - 1347

Regularization

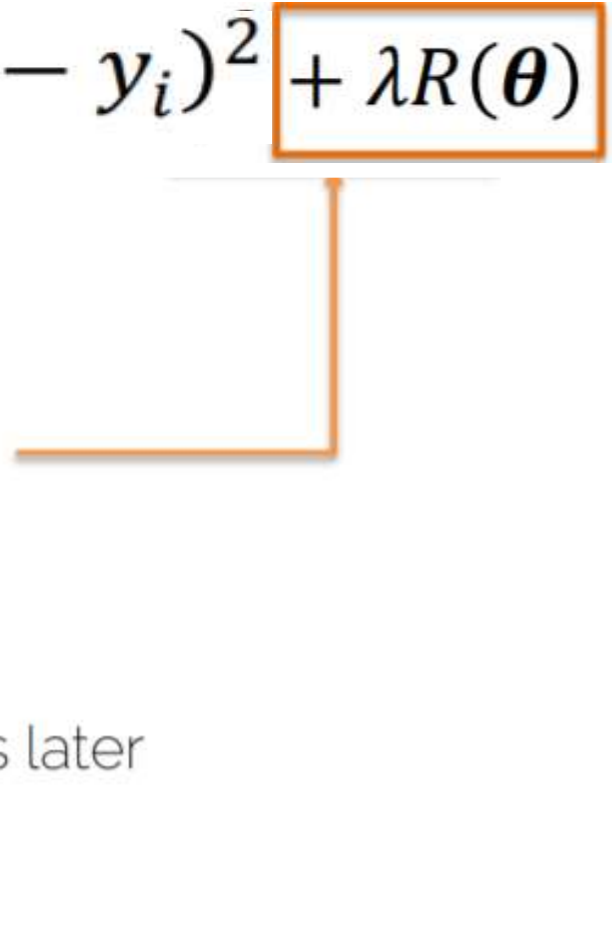
- Loss function $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\boldsymbol{\theta})$

Regularization

- Loss function $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\boldsymbol{\theta})$
 - Regularization techniques
 - L2 regularization
 - L1 regularization

Add regularization term to loss function
- 

Regularization

- Loss function $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda R(\boldsymbol{\theta})$
 - Regularization techniques
 - L2 regularization
 - L1 regularization
 - Max norm regularization
 - Dropout
 - Early stopping
 - ...
- Add regularization term to loss function
- More details later
- 

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:
- $\theta_1 = [0, 0.75, 0]$
- $\theta_2 = [0.25, 0.5, 0.25]$

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:
 - $\theta_1 = [0, 0.75, 0]$ Ignores 2 features
 - $\theta_2 = [0.25, 0.5, 0.25]$ Takes information from all features

Regularization: Example

- Loss $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (x_i \theta_{ji} - y_i)^2 + \lambda R(\boldsymbol{\theta})$

Regularization: Example

- Loss $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (x_i \theta_{ji} - y_i)^2 + \lambda R(\boldsymbol{\theta})$
- L2 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^n \theta_i^2$

Regularization: Example

- Loss $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (x_i \theta_{ji} - y_i)^2 + \lambda R(\boldsymbol{\theta})$
- L2 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^n \theta_i^2$

$$\mathbf{x} = [1, 2, 1], \theta_1 = [0, 0.75, 0], \theta_2 = [0.25, 0.5, 0.25]$$

Regularization: Example

- Loss $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (x_i \theta_{ji} - y_i)^2 + \lambda R(\boldsymbol{\theta})$
 - L2 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^n \theta_i^2$
- $\theta_1 \longrightarrow 0 + 0.75^2 + 0 = 0.5625$
- $\theta_2 \longrightarrow 0.25^2 + 0.5^2 + 0.25^2 = 0.375$ Minimization

$$\mathbf{x} = [1, 2, 1], \theta_1 = [0, 0.75, 0], \theta_2 = [0.25, 0.5, 0.25]$$

Regularization: Example

- Loss $L(\mathbf{y}, \hat{\mathbf{y}}, \boldsymbol{\theta}) = \sum_{i=1}^n (x_i \theta_{ji} - y_i)^2 + \lambda R(\boldsymbol{\theta})$

- L1 regularization $R(\boldsymbol{\theta}) = \sum_{i=1}^n |\theta_i|$

$$\theta_1 \longrightarrow 0 + 0.75 + 0 = 0.75$$

$$\theta_2 \longrightarrow 0.25 + 0.5 + 0.25 = 1$$

$$\mathbf{x} = [1, 2, 1], \theta_1 = [0, 0.75, 0], \theta_2 = [0.25, 0.5, 0.25]$$

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:

$$\theta_1 = [0, 0.75, 0]$$



L1 regularization
enforces **sparsity**

Regularization: Example

- Input: 3 features $\mathbf{x} = [1, 2, 1]$
- Two linear classifiers that give the same result:

$$\theta_1 = [0, 0.75, 0]$$



L1 regularization
enforces **sparsity**

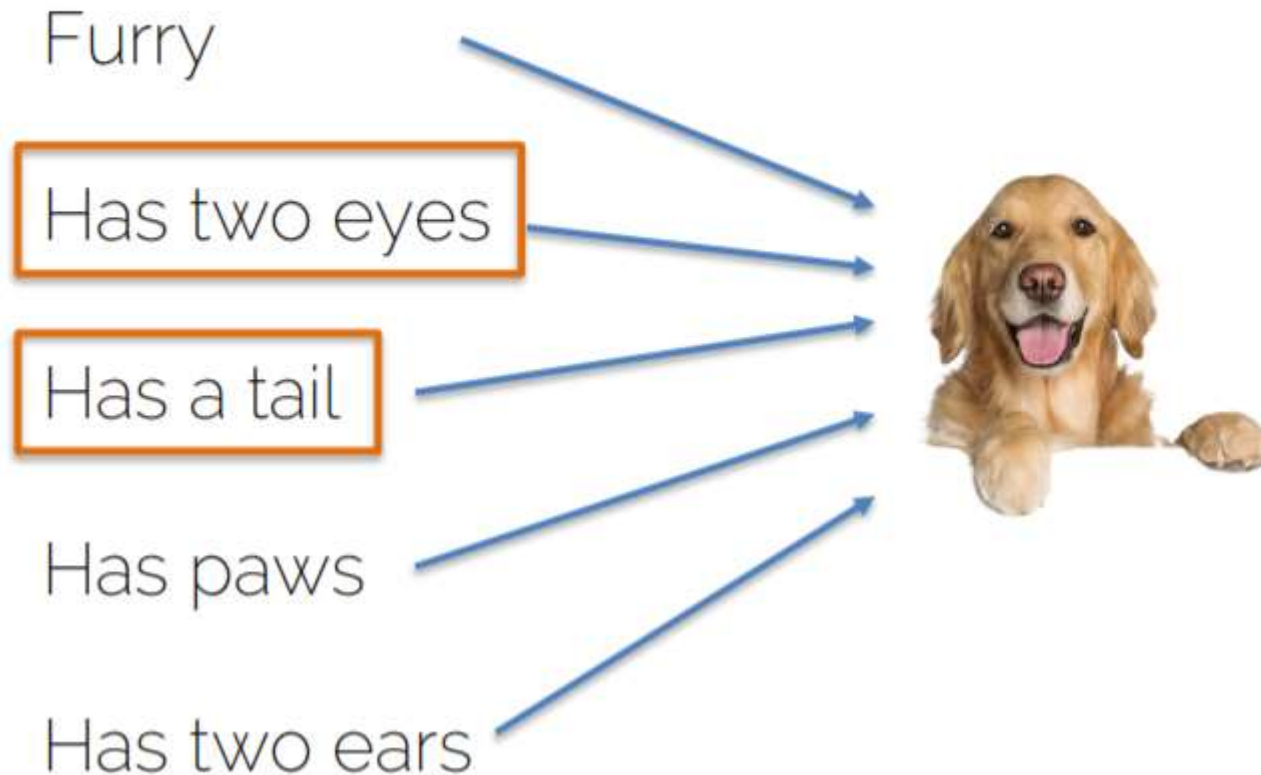
$$\theta_2 = [0.25, 0.5, 0.25]$$



L2 regularization
enforces that the weights
have **similar values**

Regularization: Effect

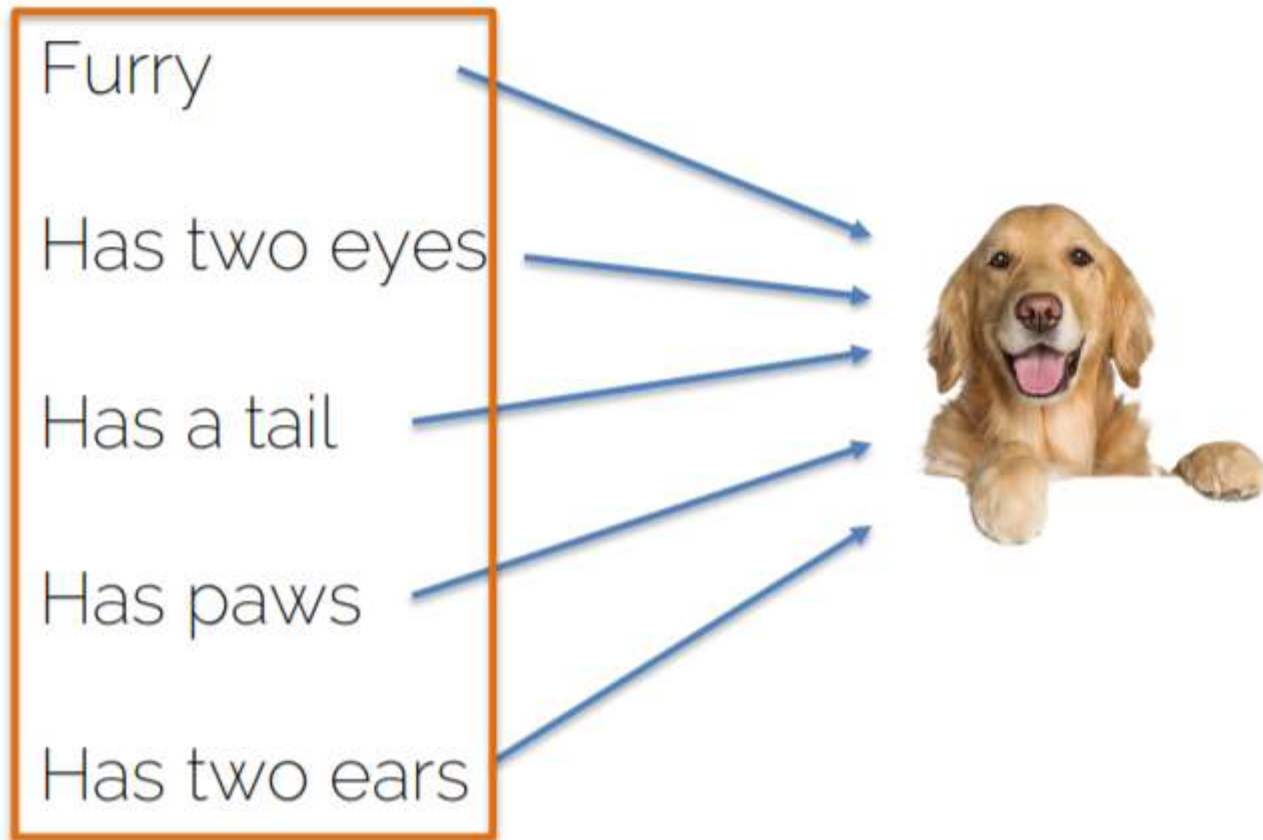
Dog classifier takes different inputs



L1 regularization
will focus all the
attention to a
few key features

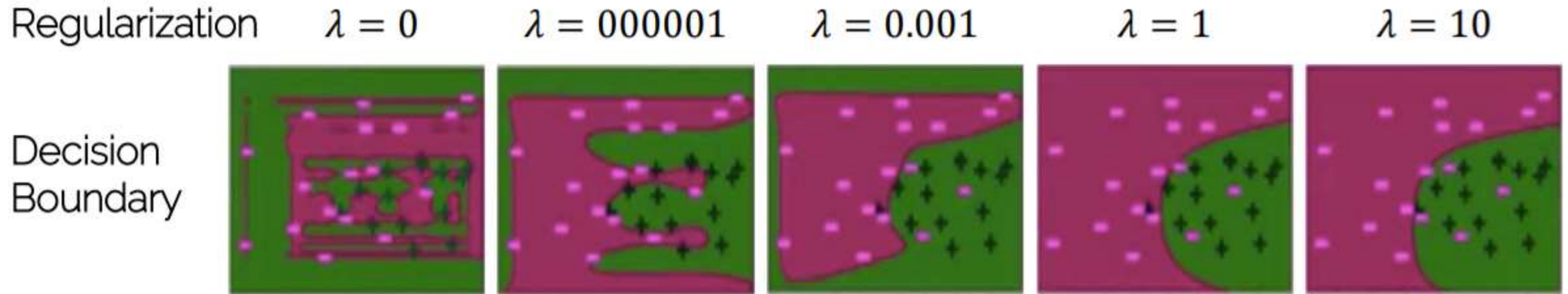
Regularization: Effect

Dog classifier takes different inputs



L2 regularization
will take all
information into
account to make
decisions

Regularization



Credits: University of Washington

Regularization

- Any strategy that aims to



Lower
validation error



Increasing
training error